

Voice agent: cost write-up and model-swap note

Round 1, local track | github.com/kishanmadhav/predixion-demo | figures computed by `cost/fargate_cost.py`

1. AWS Fargate cost estimate (us-east-1, Linux, on-demand)

Pricing: \$0.04048 per vCPU-hour and \$0.004445 per GB-hour (checked 2026-10-05), billed per second. **Load:** 10 calls/min for 22 h plus 50 calls/min for 2 h every day, i.e. 19,200 calls/day and about 584k per 730-hour month.

Concurrency: with 4-minute calls, Little's law gives 40 calls in progress at baseline and 200 at peak. **Tasks:** 1 vCPU / 2 GB (\$0.04937/h), each carrying 20 concurrent calls, because the orchestrator is I/O-bound and mostly waits on STT, LLM and TTS. One spare task for AZ redundancy gives **3 tasks at baseline and 11 at peak**.

Configuration	Fleet	Task-hours/mo	x86 \$/mo	Graviton \$/mo
A. Static, sized for peak	11 tasks, 24x7	8,030	\$396	\$317
B. Baseline + scheduled scale-out	3 tasks 24x7, +8 tasks for 2.5 h/day	2,798	\$138	\$111

The 2.5 h/day burst in B covers the 2 h campaign window, 15 minutes of pre-warm before it and 15 minutes of drain after it.

Recommendation: B. It is 65% cheaper (about \$258/month, or \$0.24 vs \$0.68 per 1,000 calls), and the spike is predictable. A scheduled scaling action raises the task minimum to 11 fifteen minutes before the window and lowers it to 3 afterwards, with CPU target tracking kept as a backstop for unplanned spikes. Reactive scaling alone would lag the 5x jump by 3-5 minutes (metric period, alarm evaluation, 30-60 s task start-up), and that lag lands exactly as the calls arrive. A deregistration delay longer than a call keeps scale-in from cutting live calls off. A only makes sense if campaign times are unpredictable. Graviton saves a further 20%. Fargate Spot does not suit live calls, because its 2-minute interruption drops them; it is fine for dead-letter replay workers.

Sensitivity. Calls per task drives the result most. At 10 calls/task, A costs \$757 and B \$240; at 40, A costs \$216 and B \$87. B wins in every case, and voice-agent loadtest measures the real figure. ALB, NAT gateway and logs (about \$50-100/month) are the same in both configurations and are excluded. Self-hosted GPU inference for STT/LLM/TTS would cost more than either fleet, since Fargate has no GPUs.

2. Swapping the mock for a real open-weight model

The resilience layer and the pipeline depend on only two things: three one-method Protocols in `providers/base.py` (`transcribe(audio) -> str`, `complete(messages) -> str`, `synthesize(text) -> bytes`) and the `ProviderError` taxonomy. `providers/factory.py` is the only code that knows which adapter backs each stage, so replacing the mock with an open-weight stack is a configuration change: set `{STT, LLM, TTS}_PROVIDER=openai` and point each `*_BASE_URL` at a server that speaks the OpenAI-compatible API, such as `Speeches` running `faster-whisper` for STT, `vLLM`, `Ollama` or `llama.cpp` serving `Qwen` or `Llama` for the LLM, and `Kokoro-FastAPI` for TTS. The `openai_compat` adapters already build those requests and classify those servers' failures (`Ollama`'s `queue-full 503` and `llama.cpp`'s `model-loading 503` are retried; `Speeches`' `403` for a bad key is not). Each stage can be swapped on its own, and `docker-compose.openweight.yml` runs the full open-weight stack on CPU by changing only environment variables. A model that runs in-process (say `faster_whisper.WhisperModel` called through `asyncio.to_thread`), or a server with a different API, needs one new adapter of about 40 lines that implements the Protocol and raises `ProviderError`; retries, circuit breakers, deadlines and dead-lettering then apply to it unchanged, with no edits to the resilience layer or the pipeline.

Verified: the adapters are contract-tested against each server's documented request and response shapes. Not run here: the open-weight compose stack, which needs multi-GB model downloads. With real models, raise `LLM_TIMEOUT_S` and `TURN_DEADLINE_S` to match measured latency.